PET Platform

Privacy-Enhancing Technologies

Rapport de developpement

Implementation des issues #6 et #7

Etudiant	Abdelmoumen Elimrany
Encadrant	Achahbi
Date	22 avril 2026
Depot	github.com/ElimranyAbdelmoumen/PET_Projet
Branche	`main` (commit `bb816b0`)

Issue #6 Support R, Microdata, Collecte des outputs

Issue #7 Navigateur securise Brave

Resume executif

Ce rapport presente l'implementation de deux issues sur la **PET Platform**, une application web Flask de soumission securisee de scripts scientifiques Python/R.

Issue #6 ajoute : execution R (`Rscript -vanilla`), selecteur de fichier microdata, et workflow de collecte/validation des fichiers de sortie.

Issue #7 securise l'acces via l'extension Brave Security Shield (MV3) : protections anti-exfiltration et enforcement HTTP 403 cote Flask.

Le **benchmark** (8 scenarios x 5 runs) confirme une latence moyenne de **1,5 secondes** sans aucune erreur sur 40 executions.

Table des matières

1 Presentation du projet **2**
- 1.1 Contexte . . . 2
- 1.2 Architecture technique . . . 2
- 1.3 Cycle de vie d'une soumission . . . 2

2 Issue #6 – Support R, microdata et collecte des outputs **2**
- 2.1 Support R dans le runner Docker . . . 2
- 2.2 Selecteur de fichier microdata . . . 3
- 2.3 Collecte et validation des fichiers de sortie . . . 4

3 Issue #7 – Navigateur securise Brave **5**
- 3.1 Architecture de l'extension (Manifest V3) . . . 5
- 3.2 Integration Flask . . . 6
- 3.3 Installation de l'extension . . . 6
- 3.4 Tests et validation en conditions reelles . . . 7

4 Benchmark du sandbox **8**
- 4.1 Protocole . . . 8
- 4.2 Resultats . . . 8
- 4.3 Analyse . . . 8

5 Conclusion **9**

Annexe – Structure du projet **10**

1 Presentation du projet

1.1 Contexte

La **PET Platform** permet a des chercheurs de soumettre des scripts de traitement de donnees pour execution dans un environnement **entierement isole**. Un administrateur valide les resultats avant qu'ils soient accessibles a l'utilisateur, garantissant que les donnees microdata sensibles ne quittent jamais le serveur.

1.2 Architecture technique

Table 1 – Composants de la PET Platform

Composant	Technologie	Version	Role
Backend	Flask + Jinja2	Python 3.11	Interface web, auth, routes
Base donnees	PostgreSQL	16	Soumissions, outputs, users
Worker	Python	3.11	Execution asynchrone
Sandbox	Docker isole	–	Conteneur ephemere par soumission
Navigateur	Brave + Extension	MV3	Acces securise, protections client

1.3 Cycle de vie d'une soumission

Table 2 – Etats d'une soumission

Etape	Etat	Description
1	`PENDING`	L'utilisateur soumet le script
2	`APPROVED`	L'administrateur approuve l'execution
3	`RUNNING`	Le worker lance le conteneur Docker
4	`FINISHED`	Execution terminee, outputs collectes
5	`REJECTED`	L'administrateur peut rejeter a l'etape 2

2 Issue #6 – Support R, microdata et collecte des outputs

Objectif : (1) Executer des scripts R en plus de Python, (2) associer un fichier microdata a chaque soumission, (3) collecter et valider les fichiers de sortie produits.

2.1 Support R dans le runner Docker

Dockerfile – ajout de R

```
# Installation de R
RUN apt-get update && apt-get install -y r-base r-base-dev \
    && rm -rf /var/lib/apt/lists/*

# Packages Python + rpy2
RUN pip install --no-cache-dir pandas numpy matplotlib \
    scikit-learn scipy seaborn rpy2

# Repertoires de sortie (droits ouverts)
RUN mkdir -p /work/outputs /work/data && chmod 777 /work/
   outputs /work/data

ENTRYPOINT ["python", "/usr/local/bin/run_script.py"]
```

Correction permissions : L'utilisateur `runner` (UID 10001) a ete supprime du Dockerfile – il empechait l'ecriture dans `/work/outputs/` car le repertoire hote etait detenu par root.

Detection Python / R dans le worker

```
# worker.py -- selection de l'interpreteur selon l'extension
ext = os.path.splitext(file_path)[1].lower() or ".py"
container_script = f"/work/script{ext}"
# .py -> exec() Python avec capture des variables
# .R  -> Rscript --vanilla /work/script.R
```

```
# run_script.py -- routing Python / R
if script_path.endswith(".R"):
    result = subprocess.run(
        ["Rscript", "--vanilla", script_path],
        capture_output=True, text=True, timeout=55
    )
else:
    # exec() avec capture automatique (result, output, df)
    ...
```

2.2 Selecteur de fichier microdata

Les fichiers microdata (CSV, Excel) sont stockes en base avec un UUID. L'utilisateur selectionne le fichier dans un menu deroulant lors de la soumission.

```
-- Schema base de donnees (db/init.sql)
CREATE TABLE microdata_files (
    id          SERIAL PRIMARY KEY,
    guid        UUID DEFAULT gen_random_uuid() UNIQUE,
    name        VARCHAR(255) NOT NULL,
    file_path   TEXT NOT NULL,
    created_at  TIMESTAMPTZ DEFAULT NOW()
);

```

```sql
ALTER TABLE submissions
    ADD COLUMN microdata_guid UUID REFERENCES
       microdata_files(guid),
    ADD COLUMN outputs_status VARCHAR(20) DEFAULT 'NONE';
```

Le fichier est monte **en lecture seule** dans le conteneur sous `/work/data/<nom_fichier>`. Exemples d'utilisation :

```python
# Python
import pandas as pd
df = pd.read_csv("/work/data/population_sample.csv")
print(df.describe())
```

```r
# R
df <- read.csv("/work/data/population_sample.csv")
cat("Lignes :", nrow(df), "\n")
print(summary(df))
```

2.3 Collecte et validation des fichiers de sortie

Table 3 – Workflow de validation des outputs

Etape	Acteur	Action
1	Script	Ecrit ses fichiers dans `/work/outputs/`
2	Worker	Scanne le repertoire local apres execution
3	Worker	Insere dans `submission_outputs` (statut PENDING_VALIDATION)
4	Administrateur	Approuve ou rejette chaque fichier
5	Utilisateur	Telecharge si APPROVED

```sql
-- Table des fichiers de sortie
CREATE TABLE submission_outputs (
    id             SERIAL PRIMARY KEY,
    submission_id INTEGER REFERENCES submissions(id) ON
       DELETE CASCADE,
    filename       VARCHAR(255) NOT NULL,
    file_path      TEXT NOT NULL,
    created_at     TIMESTAMPTZ DEFAULT NOW()
);
```

Correction du bug chemin local / chemin hote

Probleme : le worker (dans son conteneur) ne peut pas utiliser le chemin Windows de l'hote pour `os.listdir`. **Solution :** deux chemins distincts.

```python
# Chemin pour Docker Desktop (-v mount)
host_outputs_dir = os.path.join(HOST_STORAGE_PATH, "outputs
    ", str(sid))
```

```
# ex: /host_mnt/c/Users/.../storage/outputs/42

# Chemin accessible depuis le worker (volume docker-compose)
local_outputs_dir = os.path.join(STORAGE_PATH, "outputs",
    str(sid))
# ex: /storage/outputs/42

execute_script(file_path, microdata_path,
    outputs_dir=host_outputs_dir,        # pour -v
    local_outputs_dir=local_outputs_dir) # pour makedirs +
        listdir

collect_outputs(submission_id, local_outputs_dir, conn)
```

3 Issue #7 – Navigateur securise Brave

Objectif : restreindre l'acces a la plateforme au seul navigateur Brave avec l'extension Brave Security Shield (MV3) installee, et appliquer les protections cote client.

3.1 Architecture de l'extension (Manifest V3)

Table 4 – Fichiers de l'extension

Fichier	Role
`manifest.json`	Declaration MV3, permissions, hotes autorises
`content_script.js`	Protections injectees dans chaque page
`background.js`	Service worker – violations, gestion identite
`popup.html/js`	Interface admin : toggles, watermark, journal
`screen_block.css`	Overlay CSS de blocage visuel
`auth_bridge.js`	API JS pour transmettre l'identite Flask vers BSS

Table 5 – Protections implementees

Protection	Mecanisme	Actif
Copier/Coller	Interception copy, cut, paste + Clipboard API	**OK**
Capture ecran	Detection PrintScreen + overlay CSS + @media print	**OK**
Partage ecran	Remplacement de `getDisplayMedia()`	**OK**
DevTools	7 techniques : taille fenetre, F12, debugger...	**OK**
Filigrane	SVG diagonal avec UID + date, configurable	**OK**
Journal	chrome.storage.local, 200 entrees max	**OK**

3.2 Integration Flask

Detection par cookie de presence

L'extension pose le cookie `bss_active=1` (duree 24h) a chaque visite. Flask verifie ce cookie sur **toutes les routes** via `browser_required` :

```
# backend/app/utils/authz.py
def browser_required(fn):
    @wraps(fn)
    def wrapper(*args, **kwargs):
        if not request.cookies.get("bss_active"):
            # HTTP 403 + page d'installation
            return render_template("browser_required.html"),
                403
        return fn(*args, **kwargs)
    return wrapper
```

Si le cookie est absent : HTTP 403 + page d'installation de l'extension. Si present : acces autorise normalement.

Filigrane – liaison Flask / Extension

Flask injecte l'identite de l'utilisateur dans la page via Jinja2 :

```
<!-- En bas de chaque template protege -->
<script>
  var BSS_USER_ID  = {{ session.get('user_id') }};
  var BSS_USERNAME = {{ session.get('username') | tojson }};
</script>
<script type="module" src="/static/js/bss_init.js"></script>
```

```
// static/js/bss_init.js
import { initBSSAuth } from "./auth_bridge.js";
if (typeof BSS_USER_ID !== "undefined" && BSS_USER_ID) {
    initBSSAuth({ uid: String(BSS_USER_ID), displayName:
       BSS_USERNAME || null });
}
```

3.3 Installation de l'extension

1. Telecharger **Brave Browser** sur https://brave.com
2. Ouvrir `brave://extensions` et activer le mode developpeur
3. Cliquer sur « Charger l'extension non empaquetee »
4. Selectionner le dossier `breavescripts/` du projet
5. Copier l'Extension ID et le saisir dans `auth_bridge.js` (ligne 37)
6. Acceder a la plateforme : le filigrane et les protections sont actifs

L'extension est installee et configuree avec l'ID reel :

```
// breavescripts/auth_bridge.js et backend/static/js/
   auth_bridge.js
const BSS_EXTENSION_ID = "dikpdjlignemlnikaegcblghblbejfjd";
```

3.4 Tests et validation en conditions reelles

Les tests ont ete effectues dans Brave avec l'extension chargee et la plateforme en cours d'execution (`http://localhost:5000`).

Table 6 – Resultats des tests de l'Issue #7

Test	Comportement observe	Statut
Acces depuis Chrome/Firefox	Page HTTP 403 affichee, acces refuse	**OK**
Acces depuis Brave sans extension	Page HTTP 403 affichee, acces refuse	**OK**
Acces depuis Brave avec extension	Plateforme accessible normalement	**OK**
Filigrane sans connexion	Texte « CONFIDENTIEL » diagonal visible sur toutes les pages	**OK**
Filigrane apres connexion (user : abdel)	Filigrane affiche « abdel » avec date en diagonal	**OK**
Touche PrintScreen	Overlay noir 1,8 secondes, violation enregistree	**OK**
Touche F12 / DevTools	Overlay « ACCES REFUSE » avec UID et timestamp	**OK**
Ctrl+C / Ctrl+V	Toast rouge « Copier/Coller desactive »	**OK**
Clic droit	Menu contextuel desactive	**OK**

Limite documentee : les outils de capture niveau OS (Win+Shift+S, OBS, Snagit) ne peuvent pas etre bloques par une extension navigateur – cela necessite un agent systeme (C++). Cette limite est explicitement mentionnee dans la documentation du prof (`breavescripts/readme.md`).

4 Benchmark du sandbox

4.1 Protocole

Table 7 – Conditions du benchmark

Parametre	Valeur
Image Docker	`portwatch-python-runner:1.0`
Limites	512 Mo RAM, 1 CPU, reseau desactive (`-network none`)
Timeout	60 secondes par soumission
Methode	5 executions par scenario, mesure du temps total docker run
Environnement	Windows 11 Pro, Docker Desktop 4.x (WSL2)
Total	8 scenarios x 5 runs = **40 executions**

4.2 Resultats

Table 8 – Temps d'execution par scenario (ms)

Scenario	R1	R2	R3	R4	R5	Moy.
Python						
Hello world	1611	2070	1417	1689	1379	**1633**
Pandas describe (1000L)	1347	1502	1351	1271	1361	**1366**
Sklearn LinearRegression	1395	1514	1253	1484	1427	**1414**
Lecture microdata CSV	1293	1593	1395	1545	1737	**1512**
Ecriture output CSV	1570	1601	1501	1344	1398	**1482**
R						
Hello world	1613	1613	1375	1423	1344	**1473**
rnorm(1000) + stats	1743	1483	1339	1600	1484	**1529**
Lecture microdata CSV	1424	1759	1631	1464	1332	**1522**

4.3 Analyse

Demarrage dominant : la latence (1,3–1,7 s) est causee par le demarrage du conteneur Docker. Le calcul lui-meme (pandas, sklearn, R stats) prend quelques millisecondes – le sandbox isole n'introduit pas de penalite de calcul significative.

Python vs R – performances equivalentes : la difference moyenne est de **27 ms** (Python 1481 ms, R 1508 ms), en dessous de la variabilite naturelle (ecart-type 150 ms). Les deux langages sont interchangeables.

Overhead microdata/outputs negligeable : ajouter un CSV en entree ou en sortie represente moins de 10% de la latence totale.

Table 9 – Verification des contraintes

Contrainte	Resultat	Statut
Isolation reseau	Aucun paquet sortant detecte	**OK**
Memoire 512 Mo	Jamais atteinte	**OK**
CPU 1 core	Respecte	**OK**
Timeout 60 s	Moyenne 1,5 s (marge x40)	**OK**
Python	pandas, numpy, sklearn disponibles	**OK**
R	stats, utils, base disponibles	**OK**
Microdata	/work/data/ fonctionnel	**OK**
Outputs	/work/outputs/ fonctionnel	**OK**
Taux erreur	0 erreur / 40 executions	**OK**

5 Conclusion

Les deux issues ont ete integralement implementees et mergees sur `main` :

- **Issue #6** – Support R + selecteur microdata + collecte/validation des outputs avec workflow administrateur complet
- **Issue #7** – Extension Brave Security Shield : enforcement HTTP 403, filigrane utilisateur, 6 protections anti-exfiltration actives

Le benchmark confirme un cout fixe de **1,5 s** de demarrage sandbox, largement acceptable pour une plateforme scientifique. Python et R sont a performances equivalentes, et aucune des 40 executions n'a produit d'erreur.

Abdelmoumen Elimrany
22 avril 2026
github.com/ElimranyAbdelmoumen/PET_Projet

Annexe – Structure du projet

```
PET_Projet/
|-- backend/
|   |-- app/
|   |   |-- __init__.py        # Factory Flask, blueprint web
|   |   |-- routes/web.py      # Toutes les routes HTML
|   |   |-- static/js/         # auth_bridge.js, bss_init.js
|   |   |-- templates/         # login, submit, admin_*, user_view_*
|   |   +-- utils/authz.py     # login/admin/browser_required
|   +-- requirements.txt
|-- benchmark/
|   |-- run_benchmarks.sh      # 40 runs
|   +-- results.txt
|-- breavescripts/             # Extension Brave Security Shield MV3
|   |-- manifest.json
|   |-- content_script.js
|   |-- background.js
|   +-- auth_bridge.js
|-- db/init.sql                # Schema PostgreSQL
|-- docker/docker-compose.yml  # Backend + DB + Worker
|-- rapport/
|   |-- rapport.tex
|   +-- rapport.pdf
|-- runner/
|   |-- Dockerfile             # Sandbox Python + R
|   +-- run_script.py
|-- runner_worker/worker.py    # Boucle asynchrone
+-- storage/microdata/
```